LAPORAN PRAKTIKUM

POSTTEST 8

ALGORITMA PEMROGRAMAN DASAR



Disusun oleh:

Ferdika Fathur Rahman (2609106039)

Kelas (B1 ‘26)

PROGRAM STUDI INFORMATIKA

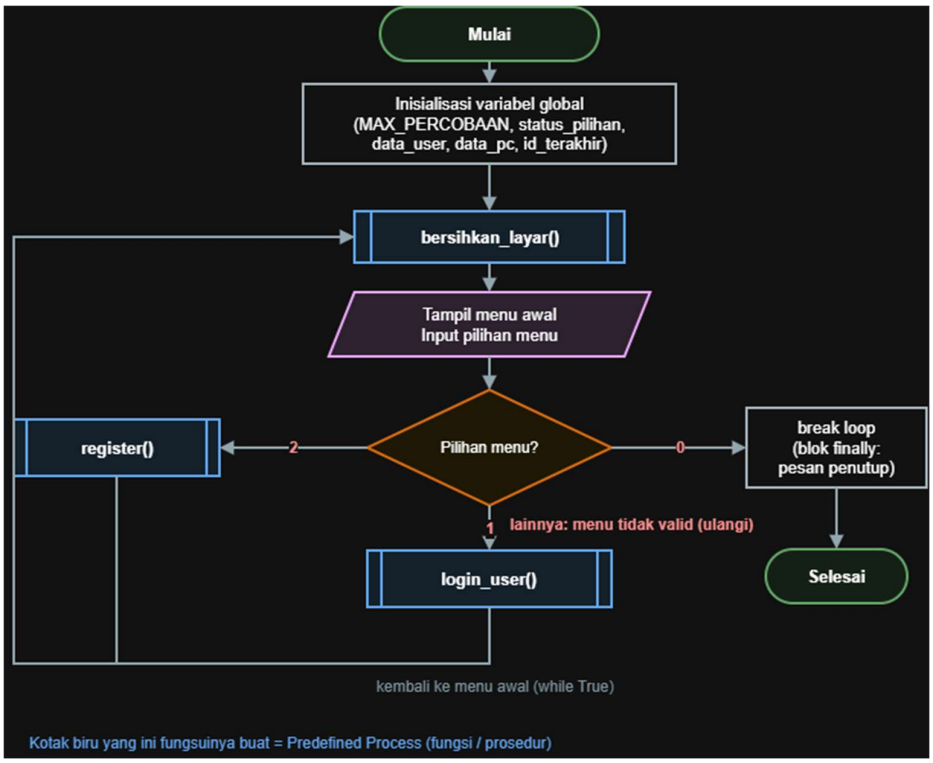
UNIVERSITAS MULAWARMAN

SAMARINDA

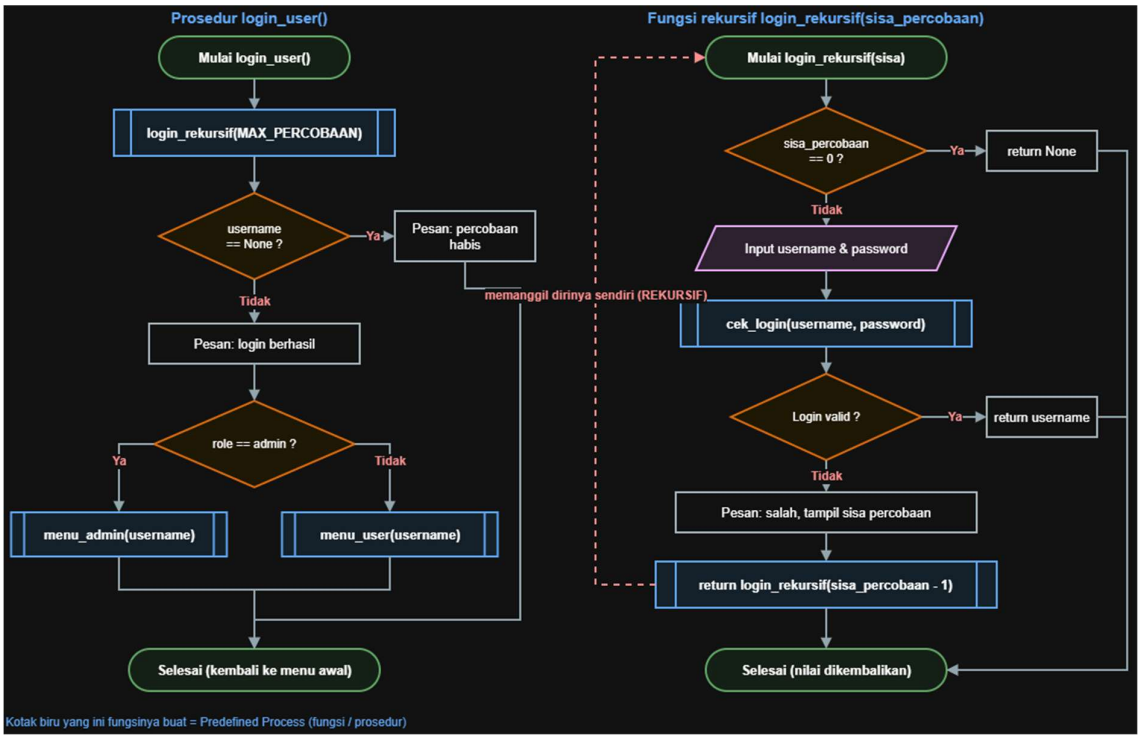
2026

1. Flowchart

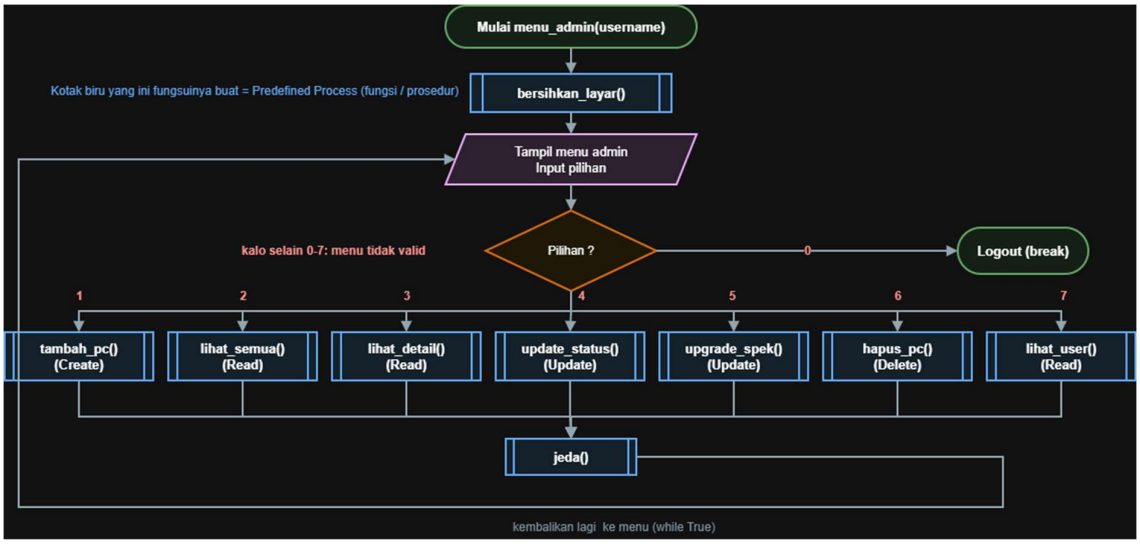
Gambar <Flowchart Menu Utama>



Gambar <Flowchart Login>



Gambar <Flowchart Menu Admin>



Penjelasan Flowchart:

Pertama, flowchart ini mulai dari menu utama yang punya 3 pilihan: login, register, sama exit. Di bagian login, kita harus masukin username dan password yang udah dibuat waktu register. Kalau salah, kita dikasih 3 kali kesempatan, dan kalau tiga-tiganya gagal, kita dibalikin ke menu utama. Di bagian register, kita bikin akun baru dengan username dan password. Username nggak boleh pakai spasi dan nggak boleh sama dengan username yang udah ada, sedangkan passwordnya minimal 4 karakter. Terakhir ada exit, yang bikin kita keluar dari sistem.

Setelah berhasil login, sistem ngecek role akun kita: admin atau pengguna biasa. Kalau masuk sebagai admin, kit dapat akses penuh buat ngelola data PC lab. Kita bisa nambah PC baru, lihat semua daftar PC, dan lihat detail plus riwayat servis tiap PC. Kita juga bisa update status PC (Tersedia, Dipakai, Rusak, atau Dalam Servis), upgrade spesifikasi (CPU, RAM, storage), dan hapus data PC. Satu fitur tambahan khusus admin: lihat daftar pengguna yang terdaftar.

Kalau masuk sebagai pengguna biasa, aksesnya lebih terbatas dan cuma buat lihat-lihat. Kita bisa lihat semua daftar PC, cek PC mana aja yang lagi tersedia, dan lihat detail serta riwayat servisnya. Nggak ada akses buat nambah, ngubah, atau ngehapus data. Di kedua role, kalau pilih 0 kita logout dan balik ke menu utama.

2. Deskripsi Singkat Program

Program ini dibuat untuk membantu pengguna mengelola dan memantau data perangkat keras di Lab Informatika, mulai dari spesifikasi, status ketersediaan, sampai riwayat servisnya. Pengguna dengan role admin memiliki hak akses tambahan, yaitu menambah data PC baru, memperbarui status kerusakan atau spesifikasi hasil upgrade, dan menghapus data aset yang sudah tidak digunakan. Program ini juga dibuat untuk menerapkan konsep CRUD (Create, Read, Update, dan Delete) dengan memanfaatkan struktur data list dan tuple.

3. Source Code

A. Fitur Login

```python
def login_rekursif(sisa_percobaan):
    # FUNGSI REKURSIF : memanggil dirinya sendiri sampai login benar
    # atau percobaan habis. Mengembalikan username / None.
    if sisa_percobaan == 0:            # base case
        return None
    username = input("Username: ").strip()
    password = input("Password: ").strip()
    if cek_login(username, password):  # base case
        return username
    print("Username atau password salah! Sisa percobaan:", sisa_percobaan - 1)
    return login_rekursif(sisa_percobaan - 1)   # rekursif
```

B. Fitur Register

```python
  def register():
    print("=== REGISTER AKUN BARU ===")
    username = input("Username (tanpa spasi): ").strip()
    password = input("Password (min 4 karakter): ").strip()
    try:
        if username == "" or password == "":
            raise ValueError("Username dan password tidak boleh kosong!")
        if " " in username:
            raise ValueError("Username tidak boleh pakai spasi!")
        if username in data_user:
            raise ValueError("Username sudah terdaftar!")
        if len(password) < 4:
            raise ValueError("Password minimal 4 karakter!")
    except ValueError as pesan_error:
        print(pesan_error)
    else:
        data_user[username] = {"password": password, "role": "user"}
        print("Register berhasil! Silakan login.")
```

C. CREATE

```python
def tambah_pc():
    print("=== TAMBAH PC BARU ===")
    nama = input_teks("Nama PC: ")
    for pc in data_pc.values():
        if pc["nama"].lower() == nama.lower():
            print("Nama PC sudah ada!")
            return
    cpu = input_teks("CPU: ")
    ram = input_angka("RAM (GB): ")
    storage = input_angka("Storage (GB): ")
    tanggal = input_tanggal("Tanggal (contoh 14-10-2026): ")
    id_baru = buat_id_baru()
    data_pc[id_baru] = {
        "nama": nama,
        "spek": {"cpu": cpu, "ram": ram, "storage": storage},
        "status": "Tersedia",
        "riwayat": {1: tanggal + " : Aset didaftarkan"}
    }
    print("Berhasil! PC", nama, "ditambahkan dengan ID". Id baru)
```

D. READ

```
def lihat_semua():
    print("=== DAFTAR SEMUA PC ===")
    tampil_tabel(data_pc)

def lihat_tersedia():
    print("=== PC YANG TERSEDIA ===")
    tersedia = {}
    for id_pc, pc in data_pc.items():
        if pc["status"] == "Tersedia":
            tersedia[id_pc] = pc
    tampil_tabel(tersedia)
    print("Total tersedia:", hitung_pc_tersedia(), "dari", len(data_pc), "PC")

def lihat_detail():
    print("=== DETAIL & RIWAYAT SERVIS ===")
    tampil_tabel(data_pc)
    id_pc, pc = cari_pc(input("\nMasukkan ID PC: ").strip())
    if pc is not None:
        print()
        tampil_detail(id_pc, pc)

def lihat_user():
    print("=== DAFTAR USER ===")
    print("Username | Role")
    print("------------------")
    for nama_user, info in data_user.items():
        print(nama_user + " | " + info["role"])
```

E. UPDATE

```
def update_status():
    print("=== UPDATE STATUS PC ===")
    tampil_tabel(data_pc)
    id_pc, pc = cari_pc(input("\nMasukkan ID PC: ").strip())
    if pc is None:
        return
    print("Status sekarang:", pc["status"])
    for nomor, nama_status in status_pilihan.items():
        print(nomor + ".", nama_status)
    try:
        status_baru = status_pilihan[input("Pilih status baru (1-4): ").strip()]
    except KeyError:
        print("Pilihan tidak valid!")
        return
    if status_baru == pc["status"]:
        print("Status sama dengan sebelumnya!")
        return
    tanggal = input_tanggal("Tanggal (contoh 14-10-2026): ")
    ket = input("Keterangan (kerusakan/servis): ").strip()
    if ket == "":
        ket = "-"
    pc["status"] = status_baru
    tambah_riwayat(pc, tanggal + " : " + status_baru + ", " + ket)
    print("Status", pc["nama"], "berhasil diubah jadi", status_baru)

def upgrade_spek():
    print("=== UPGRADE SPESIFIKASI PC ===")
    tampil_tabel(data_pc)
    id_pc, pc = cari_pc(input("\nMasukkan ID PC: ").strip())
    if pc is None:
        return
    spek = pc["spek"]
    print("Spek sekarang:", spek["cpu"], "|", spek["ram"], "GB RAM |", spek["storage"], "GB
Storage")
    print("(kosongkan kalau tidak ingin diubah)")
    cpu_teks = input("CPU baru: ").strip()
    ram_teks = input("RAM baru (GB): ").strip()
```

```
    sto_teks = input("Storage baru (GB): ").strip()
    if cpu_teks == "" and ram_teks == "" and sto_teks == "":
        print("Tidak ada yang diubah.")
        return
    try:
        cpu_baru = cpu_teks if cpu_teks != "" else spek["cpu"]
        ram_baru = int(ram_teks) if ram_teks != "" else spek["ram"]
        sto_baru = int(sto_teks) if sto_teks != "" else spek["storage"]
        if ram_baru < 1 or sto_baru < 1:
            raise ValueError("kurang dari 1")
    except ValueError:
        print("RAM dan Storage harus angka bulat minimal 1!")
        return
    tanggal = input_tanggal("Tanggal upgrade (contoh 14-10-2026): ")
    spek["cpu"] = cpu_baru
    spek["ram"] = ram_baru
    spek["storage"] = sto_baru
    tambah_riwayat(pc, tanggal + " : Upgrade jadi " + cpu_baru + " | " +
                   str(ram_baru) + " GB RAM | " + str(sto_baru) + " GB Storage")
    print("Spesifikasi", pc["nama"], "berhasil diupgrade!")
```

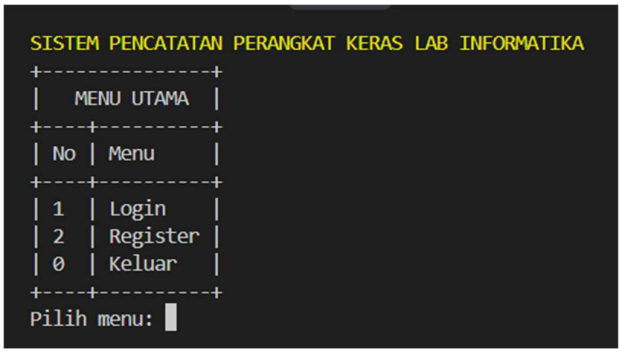
F. DELETE: hapus aset

```
def hapus_pc():
    print("=== HAPUS PC ===")
    tampil_tabel(data_pc)
    id_pc, pc = cari_pc(input("\nMasukkan ID PC: ").strip())
    if pc is None:
        return
    print("Mau hapus:", pc["nama"], "(status:", pc["status"] + ")")
    yakin = input("Aset sudah dibuang? Ketik y untuk hapus: ").strip().lower()
    if yakin == "y":
        del data_pc[id_pc]
        print("Data", pc["nama"], "berhasil dihapus!")
    else:
        print("Batal menghapus.")
```

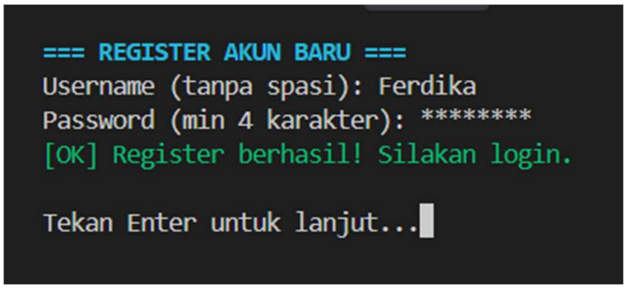
G. Lihat User

```
def lihat_user():
    print("=== DAFTAR USER ===")
    print("Username | Role")
    print("------------------")
    for nama_user, info in data_user.items():
        print(nama_user + " | " + info["role"])
```

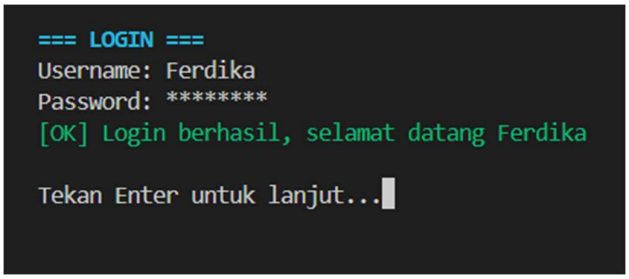
4. Hasil Output



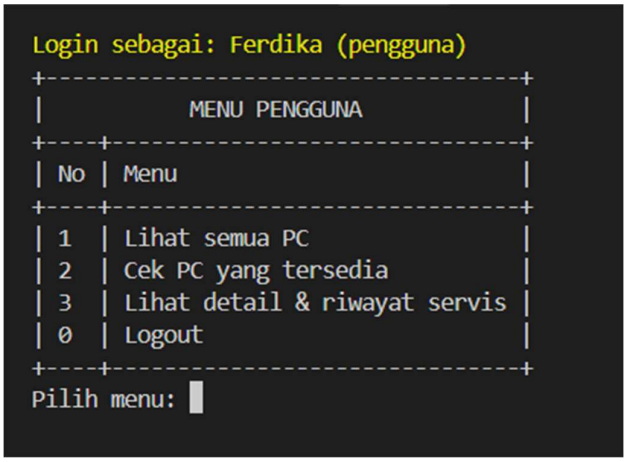
Gambar <Output Menu Utama>



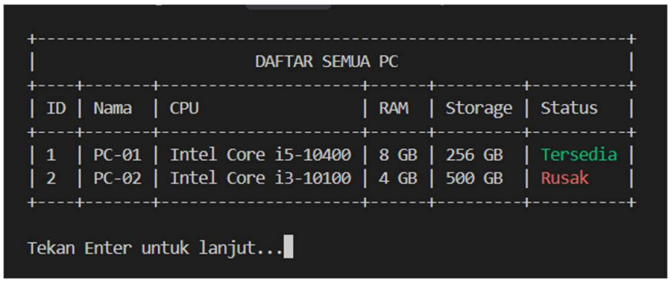
Gambar <Output Register Akun Baru>



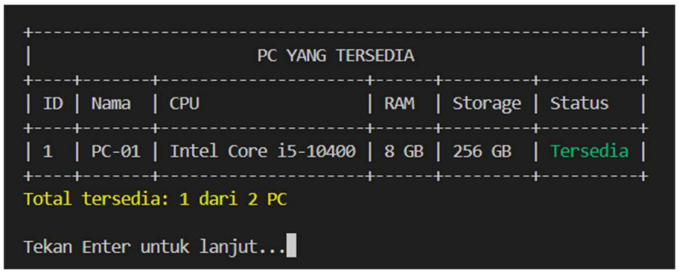
Gambar <Output Login User Baru>



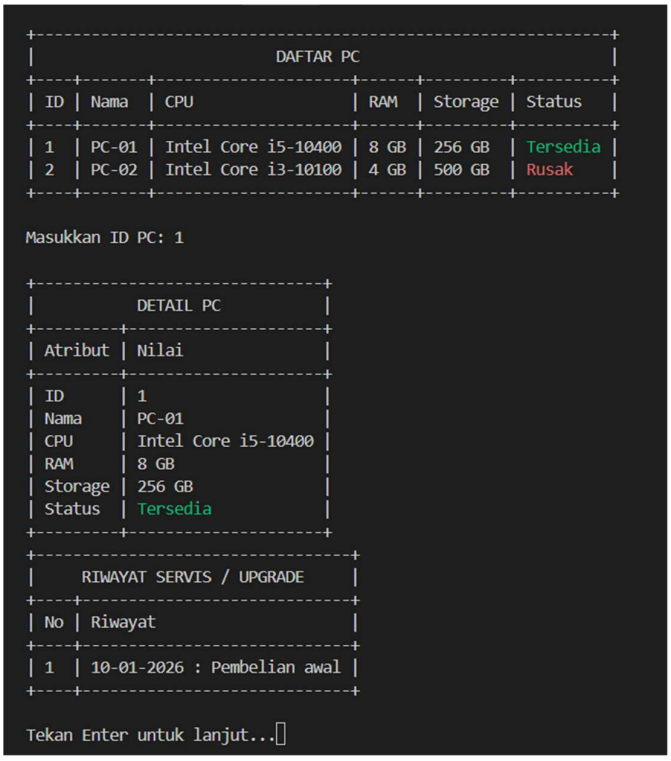
Gambar <Output Tampilan Menu Pengguna>



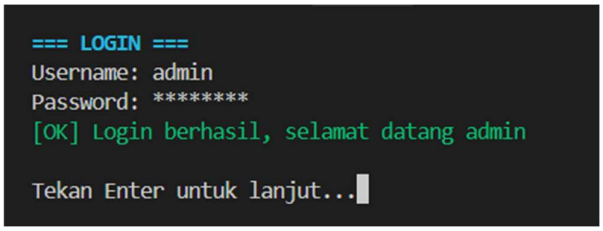
Gambar <Output Pilihan 1 User>



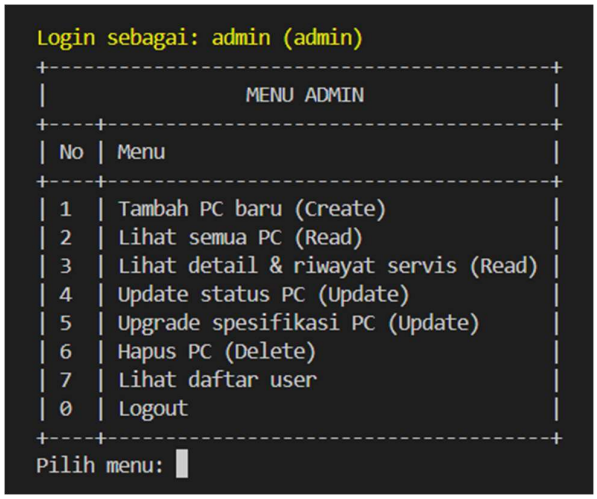
Gambar <Output Pilihan 2 User>



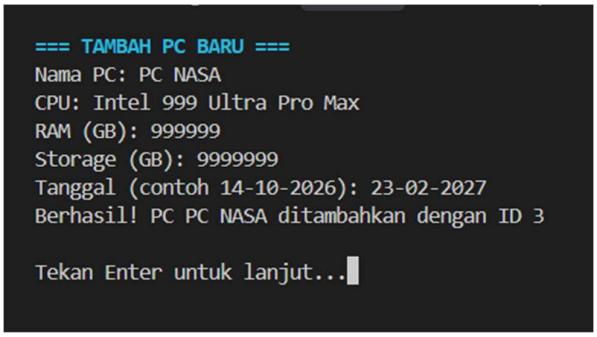
Gambar <Output Pilihan 3 User>



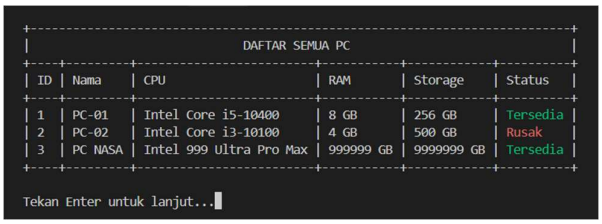
Gambar <Output Login Admin>



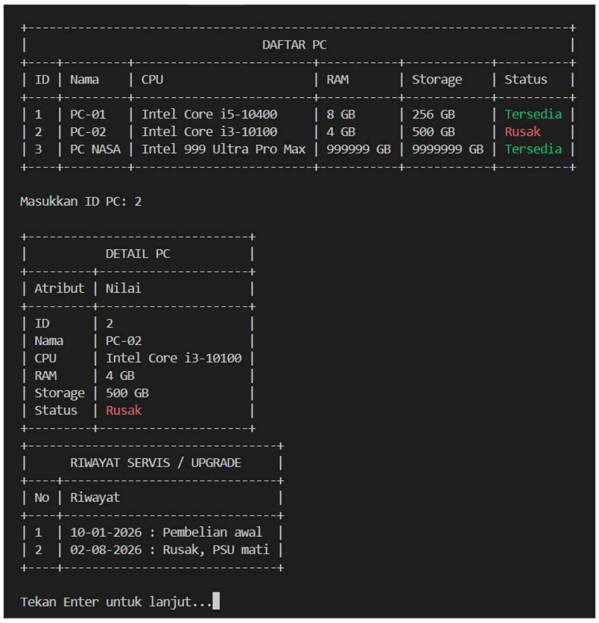
Gambar <Output Tampilan Menu Admin>



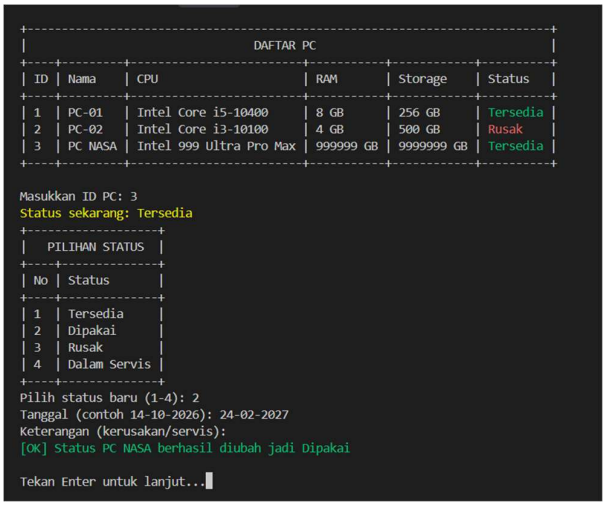
Gambar <Output Pilihan 1 CREATE Admin>



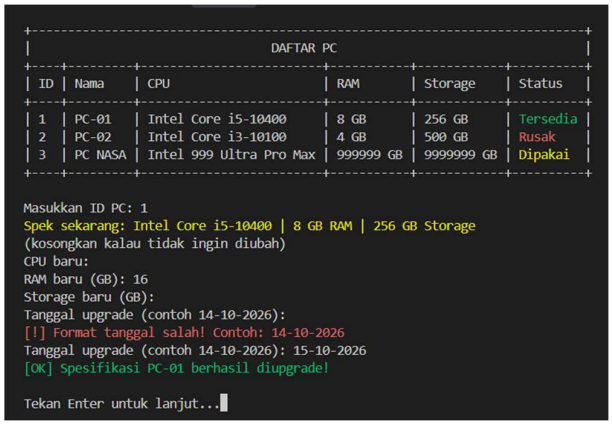
Gambar <Output Pilihan 2 READ Admin>



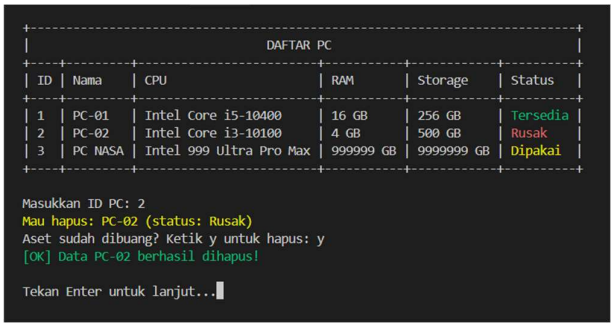
Gambar <Output Pilihan 3 READ Admin>



Gambar <Output Pilihan 4 UPDATE Admin>



Gambar <Output Pilihan 5 UPDATE Admin>



Gambar <Output Pilihan 6 DELETE Admin>



Gambar <Output Pilihan 7 Admin>

5. Git Add

Perintah “git add . “ digunakan untuk menambahkan seluruh perubahan file ke staging area git.

```
PS C:\praktikum-apd-2026\praktikum-apd-2026> git add .
PS C:\praktikum-apd-2026\praktikum-apd-2026>
```

Gambar <Hasil Perintah “git add .”>

6. Git Commit

Perintah “ git commit -m ‘teks’ “ digunakan untuk menyimpan perubahan yang sebelumnya telah dimasukkan ke dalam staging area menjadi sebuah versi atau riwayat perubahan di repository git.

```
PS C:\praktikum-apd-2026\praktikum-apd-2026> git add .
PS C:\praktikum-apd-2026\praktikum-apd-2026> git commit -m "Posttest 8 APD"
[main 98968f6] Posttest 8 APD
 8 files changed, 983 insertions(+)
 create mode 100644 posttest/posttest-apd-8/2609106039-Ferdika Fathur Rahman Fathur-PT-8.py
 create mode 100644 posttest/posttest-apd-8/crud.py
 create mode 100644 posttest/posttest-apd-8/data.py
 create mode 100644 posttest/posttest-apd-8/login.py
 create mode 100644 posttest/posttest-apd-8/main.py
 create mode 100644 posttest/posttest-apd-8/menu.py
 create mode 100644 posttest/posttest-apd-8/tampilan.py
 create mode 100644 posttest/posttest-apd-8/validasi.py
PS C:\praktikum-apd-2026\praktikum-apd-2026>
```

Gambar <Hasil Perintah “git commit -m “description””>

7. Git Push

Perintah “git push” digunakan untuk mengirimkan perubahan yang telah dicommit dari repository lokal ke repository remote, seperti GitHub.

```
PS C:\praktikum-apd-2026\praktikum-apd-2026> git push origin main
Enumerating objects: 14, done.
Counting objects: 100% (14/14), done.
Delta compression using up to 12 threads
Compressing objects: 100% (12/12), done.
Writing objects: 100% (12/12), 9.56 KiB | 2.39 MiB/s, done.
Total 12 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/runaisneat/praktikum-apd-2026.git
   bf09864..98968f6  main -> main
PS C:\praktikum-apd-2026\praktikum-apd-2026>
```

Gambar <Hasil Perintah “git push origin main”>